

✓ Break e Continue

```
for numero in range(1,10):
    if numero %2 == 0:
        continue
    print(f"Número Ímpar: {numero}")
```

```
print("Procurando o número 5...")

for numero in range(1, 11):
    print(f"Testando o número: {numero}")

    if numero == 5:
        print("Achei o 5! Parando o loop.")
        break # O break destrói o loop. O 6, 7, 8... nunca serão testados.

print("Fim da busca.")
```

✓ Estrutura de uma Função

Exemplo de uso

```
# O cliente colocou uma Camiseta no carrinho
preco_camiseta = 50.00
desconto_camiseta = preco_camiseta * 0.10
total_camiseta = preco_camiseta - desconto_camiseta

# O cliente colocou um Tênis no carrinho
preco_tenis = 200.00
desconto_tenis = preco_tenis * 0.10
total_tenis = preco_tenis - desconto_tenis

# O cliente colocou um Livro no carrinho
preco_livro = 40.00
desconto_livro = preco_livro * 0.10
total_livro = preco_livro - desconto_livro
```

```
# Você cria a função UMA VEZ:
def calcular_preco_final(preco, desconto):
    valor_do_desconto = preco * (desconto / 100)
    preco_final = preco - valor_do_desconto
    return preco_final
```

```
# O código do carrinho fica apenas assim:
total_camiseta = calcular_preco_final(50.00, 10)
total_tenis = calcular_preco_final(200.00, 10)
total_livro = calcular_preco_final(40.00, 10)
```

✓ Como usar

```
def nome_da_funcao(parametro):
    # Bloco de código
    return parametro * 2
```

```
def forjar_arma(arma, dono):
    print(f" A arma {arma} foi forjada com sucesso para {dono}!")
```

```
forjar_arma("Ferroada", "Frodo")
forjar_arma("Andúril", "Aragorn")
```

✓ Parâmetros e argumentos

```
# 'nome' é o PARÂMETRO
def saudar(nome):
    print(f"Olá, {nome}")

# 'Aragorn' é o ARGUMENTO
saudar("Aragorn")
```

```
# 'nome' é o PARÂMETRO
def saudar(nome):
    print(f"Olá, {nome}")

# 'Aragorn' é o ARGUMENTO
nome="Aragorn"
saudar(nome)
```

▼ Parâmetro padrão

```
# 'titulo' tem o valor padrão "Guerreiro"
def criar_personagem(nome, titulo="Guerreiro"):
    print(f"{nome}, o {titulo}")
criar_personagem("Legolas", "Elfo") # Saída: Legolas, o Elfo
criar_personagem("Gimli")          # Saída: Gimli, o Guerreiro
nome="Gandalf"
titulo="Mago"
criar_personagem(nome, titulo)
```

▼ Argumento Nomeado

```
def criar_personagem(nome, classe, reino):
    print(f"{nome} é {classe} de {reino}")

# Chamada usando argumentos nomeados (fora da ordem original):
criar_personagem("Gondor", "Aragorn", "Guerreiro")
criar_personagem(reino="Gondor", nome="Aragorn", classe="Guerreiro")
```

▼ Múltiplos argumentos

```
# 0 *args recebe quantos nomes forem enviados
def convocar_comitiva(*membros):
    print("Membros convocados:")
    for membro in membros:
        print(f"- {membro}")

# Chamando com 3 argumentos
convocar_comitiva("Frodo", "Sam", "Gimli")
```

```
# 0 *args recebe quantos nomes forem enviados
def convocar_comitiva(*membros):
    print("Membros convocados:")
    for membro in membros:
        print(f"- {membro}")

# Chamando com 3 argumentos
convocar_comitiva("Frodo", "Sam", "Gimli")
```

▼ Múltiplos argumentos nomeados

```
# 0 **kwargs recebe características dinâmicas
def ficha_personagem(**atributos):
    for chave, valor in atributos.items():
        print(f"{chave.capitalize()}: {valor}")

# Chamando a função com diferentes argumentos nomeados
ficha_personagem(nome="Aragorn", classe="Ranger", reino="Gondor")
```

Return

```
def enviar_mensagem(destinatario):  
    return f"Fuja, seu tolo! Mandado para: {destinatario}"
```

```
def contar_flechas(alvos atingidos):  
    total = alvos_atingidos * 2  
    return total
```

```
print(f'Legolas usou {contar_flechas(5)} flechas')
```

Variáveis locais

```
def preparar_viagem():  
    suprimentos = "Lembas" # Variável Local  
    print(f"Carregando: {suprimentos}")  
  
preparar_viagem()  
print(suprimentos)  
  
# Tentar usar 'suprimentos' fora da função vai gerar erro:  
# print(suprimentos) -> NameError!
```

Variáveis Globais

```
reino_principal = "Gondor" # Variável Global  
  
def mostrar_reino():  
    # Acessa a variável global sem problemas  
    print(f"Bem-vindo ao reino de {reino_principal}!")  
  
mostrar_reino()  
print(reino_principal) # Também acessível no programa principal
```

Alterando variáveis globais e aninhadas

```
ouro_do_reino = 100 # Variável Global  
  
def gastar_ouro(quantidade):  
    global ouro_do_reino # Permite alterar a variável global  
    ouro_do_reino -= quantidade  
  
gastar_ouro(30)  
print(f"Ouro restante: {ouro_do_reino}") # Saída: Ouro restante: 70
```

```
def comitiva():  
    membros = 9 # Variável da função externa  
  
    def membro_caiu():  
        nonlocal membros # Modifica a variável da função pai (comitiva)  
        membros -= 1  
  
    membro_caiu()  
    print(f"Membros restantes: {membros}") # Saída: 8  
  
comitiva()
```

Lambda

```
# Função tradicional (com def)  
def dobrar_dano(dano):  
    return dano * 2  
  
# Mesma lógica usando lambda
```

```
dobrar_dano_lambda = lambda dano: dano * 2

# Testando
print(dobrar_dano(50))
print(dobrar_dano_lambda(50))
```

Documentando a função

```
def calcular_dano(forca, multiplicador=1.5):
    """
    Calcula o dano final do ataque do herói.

    Parâmetros:
    forca (int): Valor base de força do herói
    multiplicador (float): Bônus da arma (padrão 1.5)

    Retorna:
    float: Dano total do ataque
    """
    return forca * multiplicador

# Consultando a documentação da função no terminal/IDE
help(calcular_dano)
```

Recursividade

```
# Contagem regressiva para a explosão do Anel na Montanha da Perdição
def contagem_regressiva(tempo):
    # Caso Base: para a recursão
    if tempo <= 0:
        print(" O Anel foi destruído!")
        return

    # Passo Recursivo: imprime e chama a função com tempo - 1
    print(f"Faltam {tempo} segundos...")
    contagem_regressiva(tempo - 1)

# Chamando a função
contagem_regressiva(3)
```